Testování aplikace pro objednávání zmrzliny

Michal Hrouda

v0.1.0
15. 4. 2026

Obsah

TODO List . . . 2

Úvod . . . 2

Zadání webové aplikace . . . 2

Popis funkcionality aplikace . . . 2

Testovací strategie . . . 2
- Uživatelská role . . . 2
  - Testovací účty . . . 2
- Zákaznická část . . . 3
- Administrátorská část . . . 3
- Prioritizace částí aplikace . . . 3

Test levels . . . 3

Testy vstupů . . . 3
- Konfigurátor zmrzliny . . . 3
  - Přehled vstupů formuláře . . . 3
  - Třídy ekvivalence . . . 4
  - Pairwise testování konfigurátoru . . . 6
    - Faktory a hodnoty . . . 6
    - Pairwise matice — 9 testovacích případů . . . 6

Testy průchodů . . . 7
- Popis procesu . . . 7

Implementace testů . . . 8

TODO List

1. Doplnit popis aplikace
2. Doplnit screenshoty aplikace
3. Dopsat testovací strategii
4. Opravit třídy ekvivalence

Úvod

Zadání webové aplikace

Testovaná webová aplikace byla vygenerována pomocí nástroje **Claude Opus 4**. Zadání znělo: vytvořit e-commerce webovou aplikaci postavenou na frameworku **Spring Boot**, která umožní zákazníkům online objednávání zmrzliny z více zmrzlináren.

Aplikace měla splňovat následující požadavky:

- výběr pobočky (zmrzlinárny) a přidání produktů do košíku
- konfigurátor vlastní zmrzliny (výběr základu, kopečků, toppingů a sušenek)
- možnost **registrace a přihlášení** zákazníka
- přehled vlastních objednávek pro přihlášeného uživatele
- **administrátorská sekce** pro správu objednávek, produktů a obchodů
- sledování stavu zásob produktů

Popis funkcionality aplikace

Webová e-commerce aplikace pro objednávání zmrzliny online. Zákazník si vybere pobočku, přidá produkty do košíku nebo si sestaví vlastní zmrzlinu pomocí konfigurátoru, a odešle objednávku k vyzvednutí.

Testovací strategie

Uživatelská role

Uživatel může nabývat tří rolý **nepřihlášený uživatel, přihlášený uživatel a admin**, kde každá role přidává uživateli nové funkcionality

- Nepřihlášený uživatel = možnost objednání zmrzliny, konfigurátor zmrzliny, filtrace nabýdky a kvíz
- Přihlášený uživatel = uchovávání objednávek, předvyplnění informací u objednávání
- Admin = dashboard, správá produktů a kamenných prodejen

Testovací účty

- Admin
  Uživatelské jméno: **admin**
  Heslo: **heslo123**

Zákaznická část

Stránka	URL adresa
Hlavní stránka	`/`
Detail zmrzlinárny	`/shop/{id}`
Košík	`/cart`
Konfigurátor zmrzliny	`/configurator`
Pokladna	`/order/checkout`
Potvrzení objednávky	`/order/confirmation`
Moje objednávky	`/order/my-orders`
Registrace	`/register`
Login	`/login`

Administrátorská část

Stránka	URL adresa
Dashboard	`/admin`
Správa zmrzlináren	`/admin/shops`
Správa produktů	`/admin/products`
Správa objednávek	`/admin/orders`

Prioritizace částí aplikace

Test levels

Testy vstupů

Konfigurátor zmrzliny

Jako testovaný formulář byl zvolen **konfigurátor vlastní zmrzliny** (`/configurator`). Jedná se o pětikrokový formulář, který umožňuje zákazníkovi sestavit zmrzlinu výběrem zmrzlinárny, obalu, počtu a příchutí kopečků, posypek a sušenek. Výsledná cena je počítána na serveru. Formulář je netriviální — kombinuje povinné i volitelné vstupy, číselná omezení i omezení výčtem hodnot.

Přehled vstupů formuláře

Pole	Typ	Povinné	Omezení
`shopId`	radio	ano	musí existovat v systému
`container`	radio	ano	`"kornout"` nebo `"kelimek"`

Pole	Typ	Povinné	Omezení
`flavors`	select (1–5×)	ano	min 1, max 5; hodnota z předdefinovaného seznamu 15 příchutí
`selectedToppings`	checkboxy	ne	0–6; hodnoty z předdefinovaného seznamu
`selectedCookies`	checkboxy	ne	0–5; hodnoty z předdefinovaného seznamu

Třídy ekvivalence

Zelené řádky = platná třída, červené = neplatná třída.

ID	Pole	Třída	Popis	Výsledek
E1	`shopId`	platná	existující ID aktivní zmrzlinárny (např. `"shop-1"`)	OK — formulář se zpracuje
E2	`shopId`	neplatná	neexistující nebo prázdné ID	chyba: „Vybraná zmrzlinárna neexistuje.„
E3	`container`	platná	`"kornout"` — cena 15 Kč	OK — použije se cena kornoutu
E4	`container`	platná	`"kelimek"` — cena 10 Kč	OK — použije se cena kelímku
E5	`container`	neplatná	jiná hodnota (mimo výčet)	tiše použije cenu kelímku (`isCone = false`)
E6	`flavors`	platná	1 kopeček — hraniční minimum	OK — cena = základ + 25 Kč
E7	`flavors`	platná	2–4 kopečky — střední rozsah	OK — cena odpovídá počtu
E8	`flavors`	platná	5 kopečků — hraniční maximum	OK — cena = základ + 125 Kč
E9	`flavors`	neplatná	0 kopečků (pole zcela chybí)	HTTP 400 — povinný parametr
E10	`flavors`	neplatná	6+ kopečků (obejití UI limitu)	server akceptuje — chybí max. validace
E11	`flavors`	neplatná	příchuť mimo předdefinovaný seznam	server akceptuje — chybí validace výčtu

ID	Pole	Třída	Popis	Výsledek
E12	`toppings` / `cookies`	platná	žádný výběr (pole vynecháno)	OK — bez příplatku
E13	`toppings` / `cookies`	platná	1 až max−1 položek z výčtu	OK — cena navýšena
E14	`toppings` / `cookies`	platná	přesně max položek (toppings: 6, cookies: 5) — hraniční maximum	OK — cena navýšena
E15	`toppings` / `cookies`	neplatná	počet položek nad maximum (obejití UI limitu)	server akceptuje — chybí max. validace
E16	`toppings` / `cookies`	neplatná	hodnota mimo předdefinovaný seznam	server akceptuje — chybí validace výčtu
E17	stav košíku	platná	košík je prázdný	OK — položka přidána
E18	stav košíku	platná	košík obsahuje produkty ze **stejné** zmrzlinárny	OK — položka přidána
E19	stav košíku	neplatná	košík obsahuje produkty z **jiné** zmrzlinárny	chyba: „Košík obsahuje produkty z jiné zmrzlinárny.„

Pairwise testování konfigurátoru

Technika **pairwise testing** (all-pairs) redukuje počet kombinací tak, aby každá dvojice hodnot libovolných dvou vstupních parametrů byla pokryta alespoň jedním testovacím případem. Namísto úplného kartézského součinu (jenž by u 4 faktorů s hodnotami $2 \times 3 \times 3 \times 3 = 162$ kombinací) stačí 9 testů.

Faktory a hodnoty

Faktor	Hodnoty	Poznámka
A — `container`	`"kornout"`, `"kelimek"`	2 hodnoty — výčtový typ
B — kopečky	1, 3, 5	min, střed, max povoleného rozsahu
C — posypky	0, 3, 6	žádná, střed, max (6)
D — sušenky	0, 2, 5	žádná, střed, max (5)

Pairwise matice — 9 testovacích případů

Každý pár hodnot libovolných dvou faktorů se v tabulce vyskytuje alespoň jednou.

cena = (kornout/kelímek) + (počet kopečků) × 25 + (počet posypek) × 10 + (počet sušenek) × 15 (Kč)

ID	kornout/kelímek	kopečky	posypky	sušenky	cena (Kč)
PW1	`kornout`	1	0	0	$15 + 25 = \mathbf{40}$
PW2	`kornout`	3	3	2	$15 + 75 + 30 + 30 = \mathbf{150}$
PW3	`kornout`	5	6	5	$15 + 125 + 60 + 75 = \mathbf{275}$
PW4	`kelimek`	1	3	5	$10 + 25 + 30 + 75 = \mathbf{140}$
PW5	`kelimek`	3	6	0	$10 + 75 + 60 = \mathbf{145}$
PW6	`kelimek`	5	0	2	$10 + 125 + 30 = \mathbf{165}$
PW7	`kornout`	1	6	2	$15 + 25 + 60 + 30 = \mathbf{130}$
PW8	`kelimek`	3	0	5	$10 + 75 + 75 = \mathbf{160}$
PW9	`kornout`	5	3	0	$15 + 125 + 30 = \mathbf{170}$

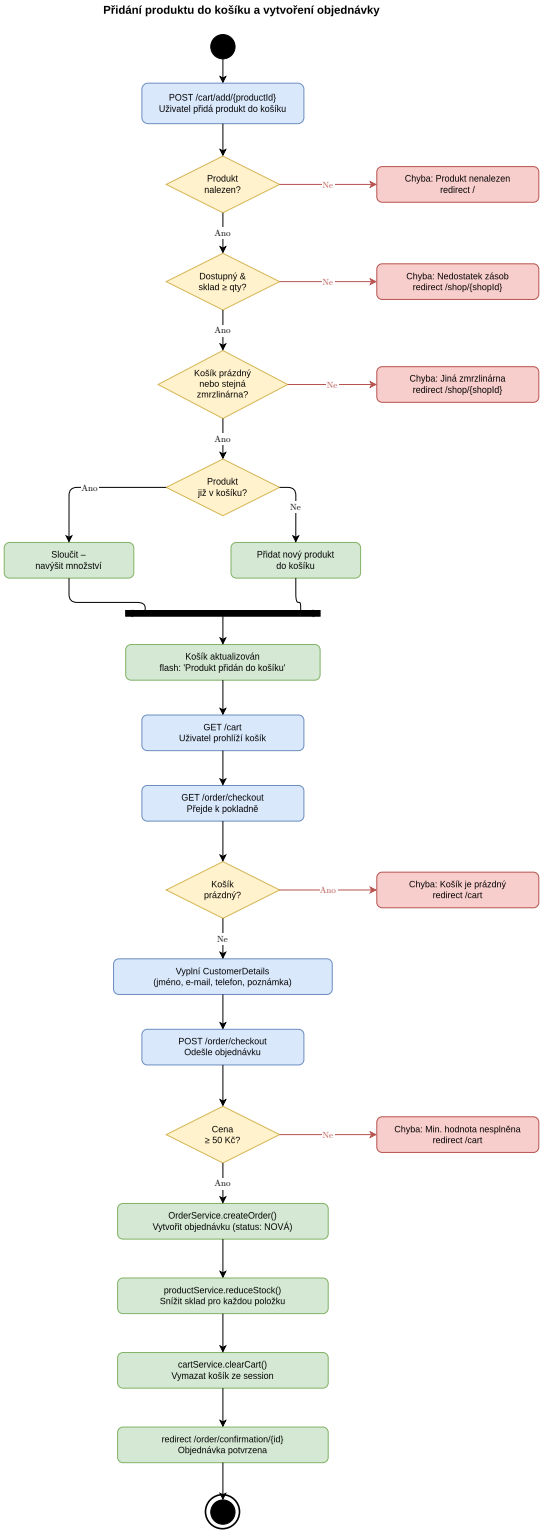
Testy jsou implementovány jako JUnit 5 parametrizovaný test (`@ParameterizedTest` + `@MethodSource`) ve třídě `ConfiguratorControllerPairwiseTest` a využívají `@WebMvcTest` s mockovanými závislostmi `ShopService` a `CartService`. Pro každý případ se ověřuje, že:

- odpověď serveru je přesměrování na `/configurator` (HTTP 3xx),
- cena zachycená v `CartItem` odpovídá očekávané hodnotě.

Testy průchodů

Popis procesu

Testovaný proces pokrývá **přidání produktu do košíku** (`POST /cart/add/{productId}`) a navazující **checkout** (`POST /order/checkout`). Diagram níže zachycuje průchod jako UML Activity Diagram s **6 rozhodovacími uzly**.



Implementace testů